

How to effectively use embeddings in tree based models

PyData Global 2025

Claudio Salvatore Arcidiacono
Machine Learning Engineer @ Mollie

Problem Definition

- We have a machine learning problem with **multimodal** input variables.
- We want to solve this problem using a **tree based** model.

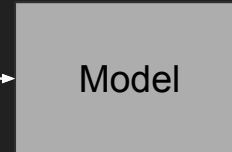
Example Use Case: House pricing estimation

Unstructured Data:
Photos
Description



The photo shows a white house with a red 'HOUSE FOR SALE' sign in the foreground. The house has a gabled roof and a small porch. The sign is red with white text. The house is surrounded by green grass and some shrubs.

Structured data:
Square Meters
of bedrooms
Postcode



Selling Price

What are Embedding models?



Unstructured Data

The best letter for your friend. Dream big. Think positive, feel positive, live positive. The future depends on what you do today. Do what you love, love what you do. If you can dream it, you can do it. Don't fear your life! Think outside the box. I go, deciding on your future with your past. Life is what you make it. Follow your dreams, they know the way. Never give up! It's go on an adventure! Wake up and smile. Attitude is a little thing that makes a big difference. Train your mind to see the good in everything. When it rains look for rainbows, when it's dark look for stars. There is time wisdom's best. Successful people never worry about what others are doing. The secret of your future is hidden in your daily routine. Life is short and the world is wide. Be your own kind of beautiful. It is during our darkest moments that we must focus to see the light. Be the woman that someone smiles today. There is always something to be thankful for. Look at all of the beauty in your life and be happy. Don't limit your imagination, challenge your limits. Every day is a chance to get better, don't waste it. Dream big, work hard, stay focused on your dreams with good people. Don't be afraid to just be afraid and to try, stop thinking that thing. Failure is the best teacher. You are stronger than you think. The best time comes after the hardest trial. Do it with passion or not at all. Find today is the hardest. Just make it! Today is a perfect day to start doing your dreams. Enjoy every moment! Be in harmony with yourself. Trust your balance. Happy people focus on what they have. Unhappy people focus on what is missing. Relax and enjoy life. Take it easy. Don't worry be happy.

Embedding Model

1.25
0.68

.

.

.

.

.

.

.

.

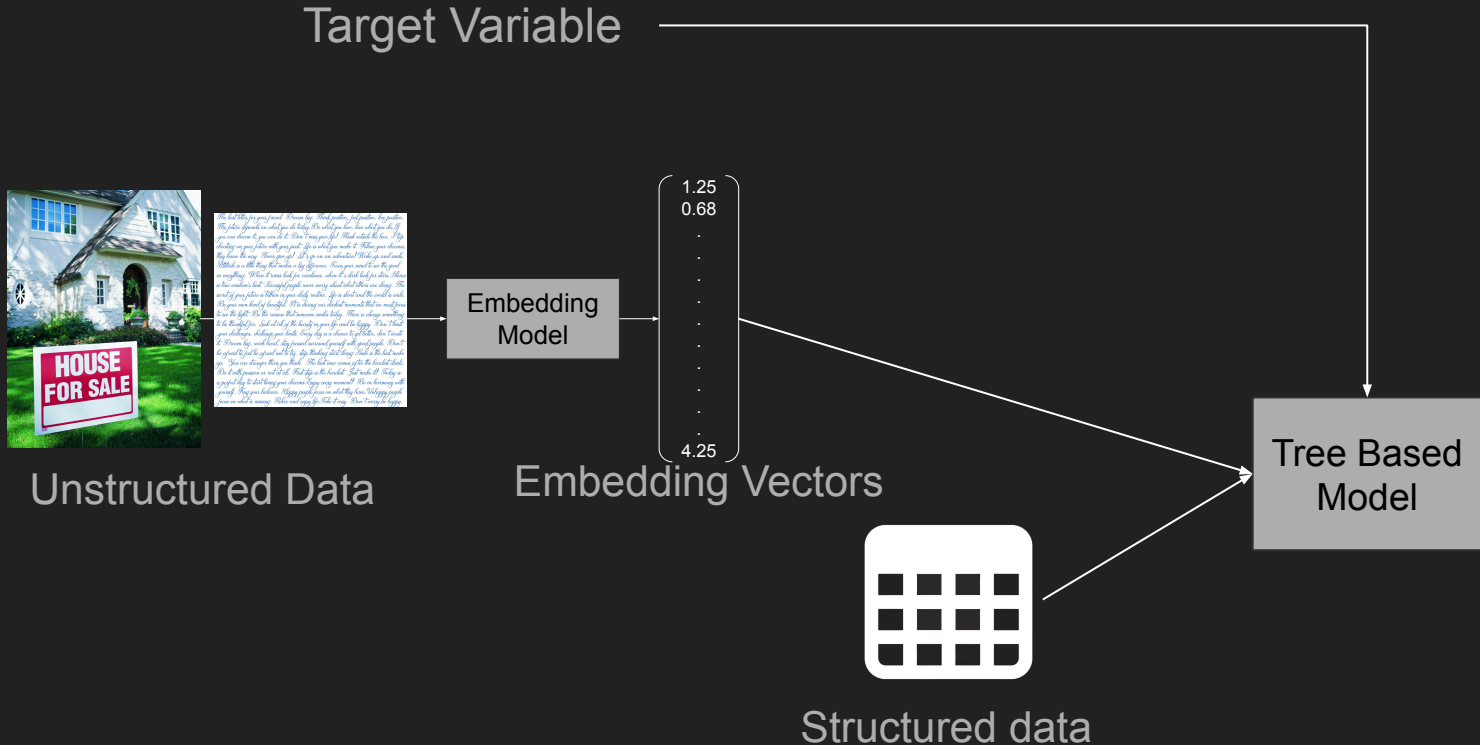
.

.

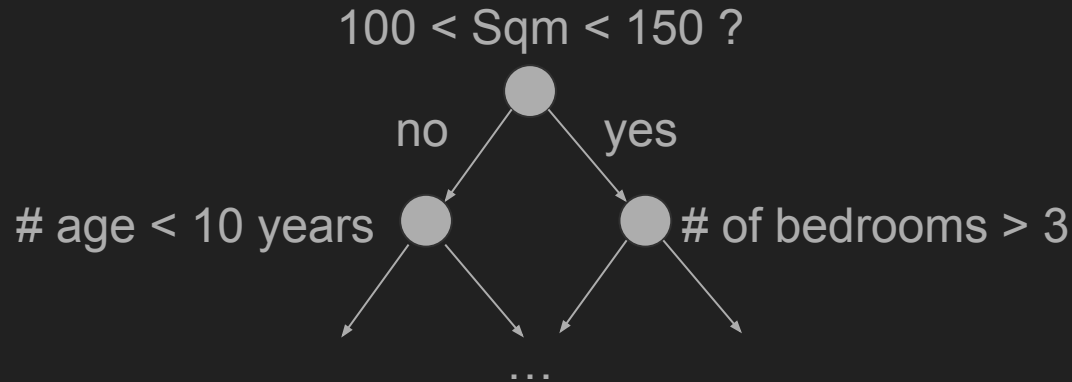
4.25

Embedding Vectors
Structured Data

First Possible Solution



Tree based models



The number of features a single tree can use is limited to the depth of the tree.

Embedding Vector

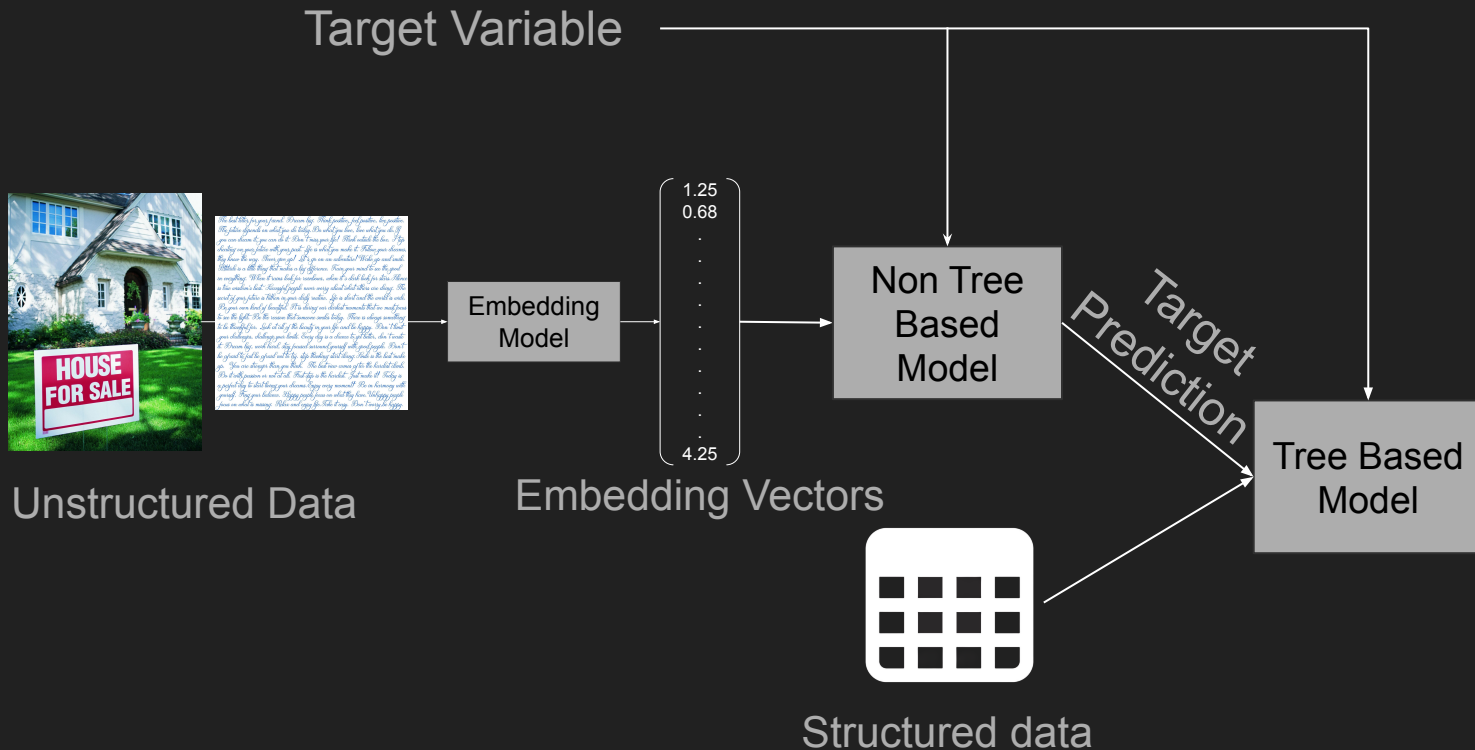
- Hundreds of dimensions (features).
- Each dimension is a numerical value.
- Multiple dimensions needs to be used simultaneously to capture the essence of the unstructured data relevant to our problem.

A decision tree needs multiple splits in order to extract information from this kind of features.

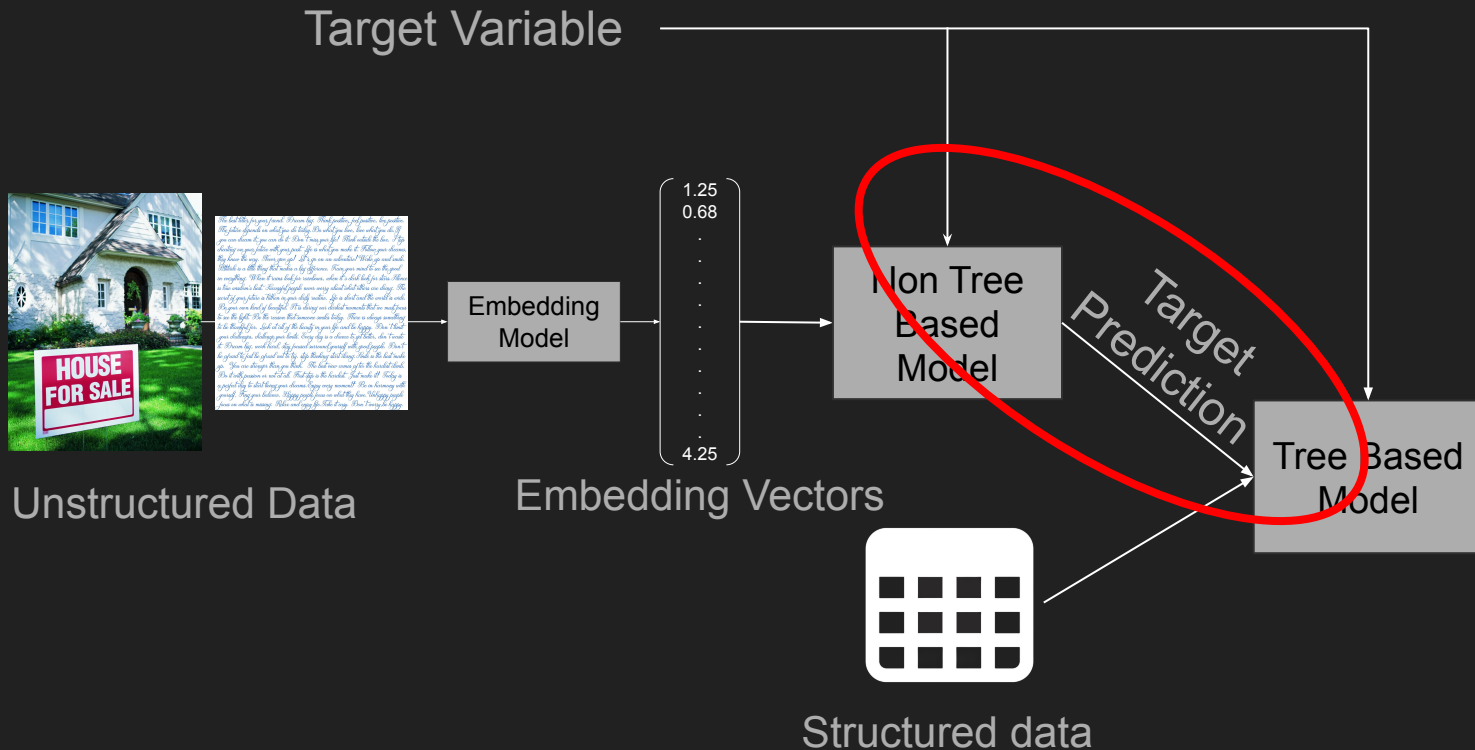
$$\begin{pmatrix} 1.25 \\ 0.68 \\ . \\ . \\ . \\ . \\ . \\ . \\ . \\ . \\ 4.25 \end{pmatrix}$$

How to use embeddings in tree based models?

Ensembling Non Tree Based models



Target Leakage?



Cross Fitting

Cross Fitting

Cross fitting is a technique used in the **stacking** ensemble technique and often used in **target encoding** of categorical features.

In our context, cross fitting can be used to train the non decision tree model on the embeddings using the “train” folds and to predict using the “validation” fold.

This way the prediction will be obtained without using the target variable of the same fold.

Embeddings

Target

Prediction

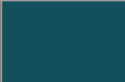
Embeddings

Target

Train

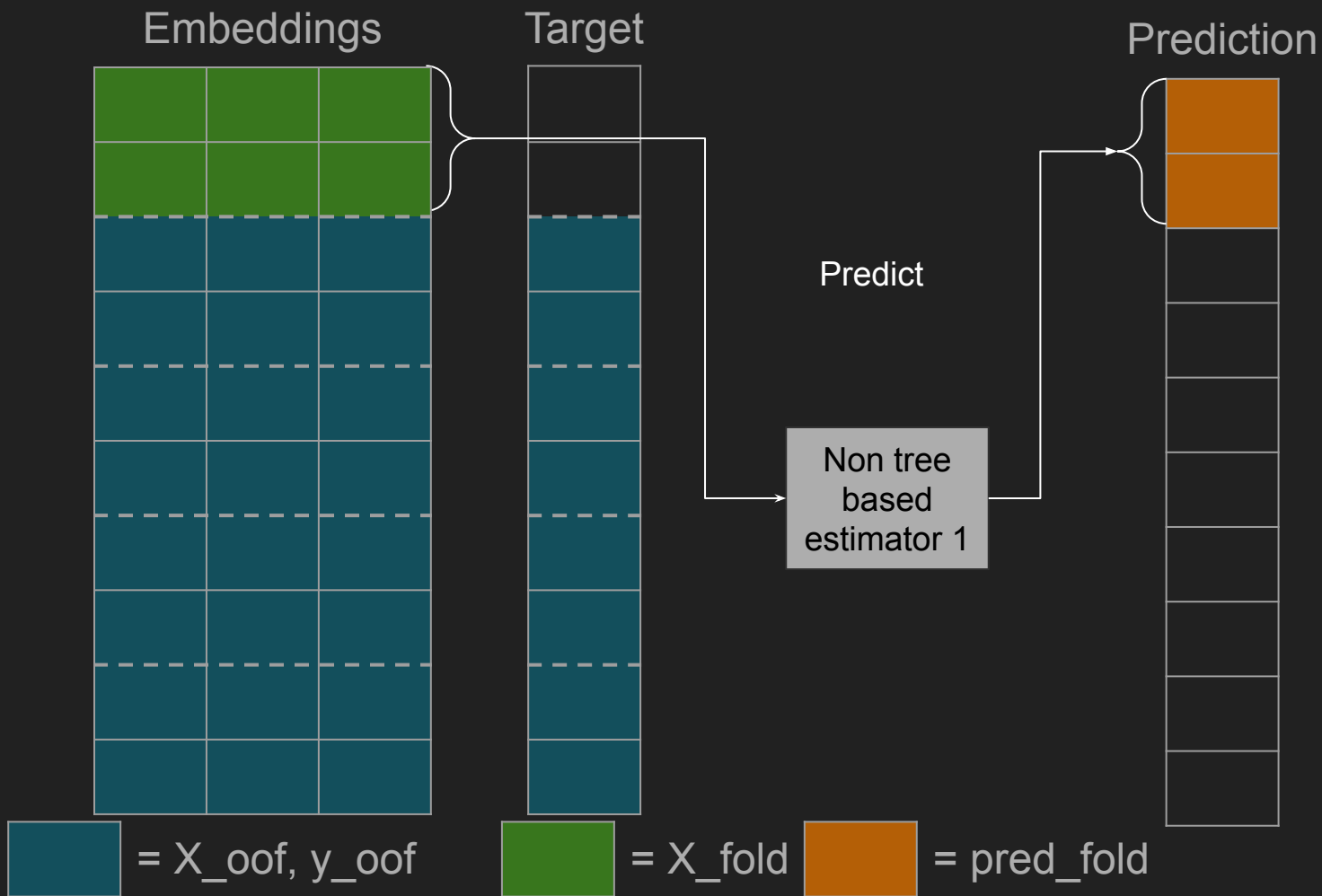
Non tree
based
estimator 1

Prediction

 = $X_{\text{oof}}, y_{\text{oof}}$

 = X_{fold}

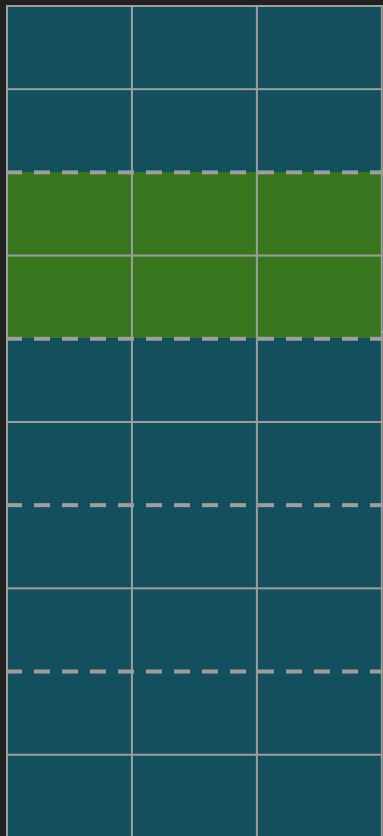
 = pred_fold



Embeddings

Target

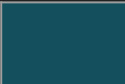
Prediction



Predict

Non tree
based
estimator 2



 = $X_{\text{oof}}, y_{\text{oof}}$

 = X_{fold}

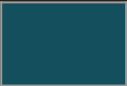
 = $\text{pred}_{\text{fold}}$

Embeddings

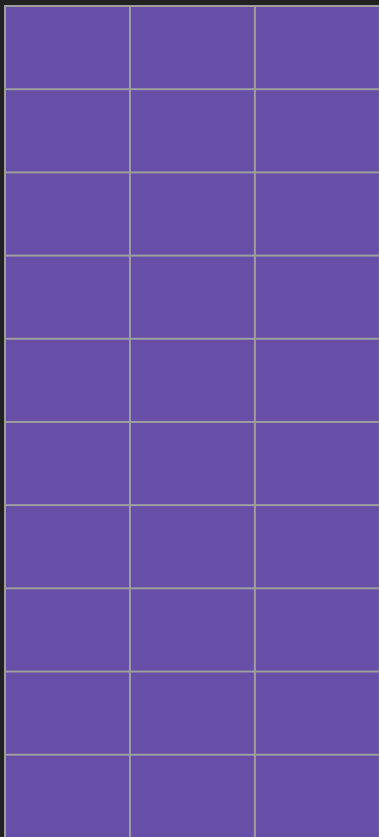
Target

Train

Non tree
based
estimator

 = $X_{\text{train}}, y_{\text{train}}$

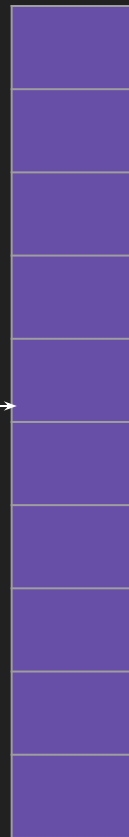
Embeddings

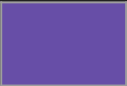


Predict

Non tree
based
estimator

Prediction



 = X_{test}

Example Implementation using scikit-learn

Imports

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import StackingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.pipeline import Pipeline
```

Sample Data

```
df = pd.DataFrame(  
    {  
        "square_meters": [50, 75, 100],  
        "number_of_rooms": [2, 3, 4],  
        "embedding_dimension_1": [0.1, 0.2, 0.3],  
        "embedding_dimension_2": [0.4, 0.5, 0.6],  
        "embedding_dimension_3": [0.7, 0.8, 0.9],  
        "price": [200_000, 300_000, 400_000],  
    }  
)
```

Embedding Processor Model

```
linear_model_pipeline = Pipeline(  
    steps=[  
        (  
            "preprocessor",  
            ColumnTransformer(  
                transformers=[  
                    (  
                        "embedding_features",  
                        "passthrough",  
                        make_column_selector(pattern="embedding_*"),  
                    ),  
                ],  
            ),  
        ),  
        ("regressor", LinearRegression()),  
    ]  
)
```

Tree based model

```
random_forest_model_pipeline = Pipeline(  
    steps=[  
        ("preprocessor",  
         ColumnTransformer(  
             transformers=[  
                 (  
                     "numerical_features",  
                     "passthrough",  
                     [0, 1, 2] # issue #16473 of scikit-learn  
                 ),  
            ],  
         ),  
        ("regressor", RandomForestRegressor(n_estimators=10, random_state=42)),  
    ]  
)
```

Stacking Model

```
stacking_model = StackingRegressor(  
    estimators=[  
        ("linear_model", linear_model_pipeline),  
    ],  
    final_estimator=random_forest_model_pipeline,  
    passthrough=True,  
    cv=3,  
)  
  
X = df.drop(columns=["price"])  
y = df["price"]  
  
stacking_model.fit(X, y)
```

Real life Use-case

- **Problem:** Webshop Classification.
- **Structured Data:** Firmographic information.
- **Unstructured Data:** Webshop text content.
- **Performance Metric:** A custom metric similar to a weighted PR AUC.

No Embedding data	Embeddings used directly	Embedding transformed via Stacking
66%	67% [▲ 1%]	69% [▲ 3%]

Thank you for your attention!